

Lecture 2

Developing Android Apps

Dr. Murad Mustafa Badarna

How to design your activity look And Feel

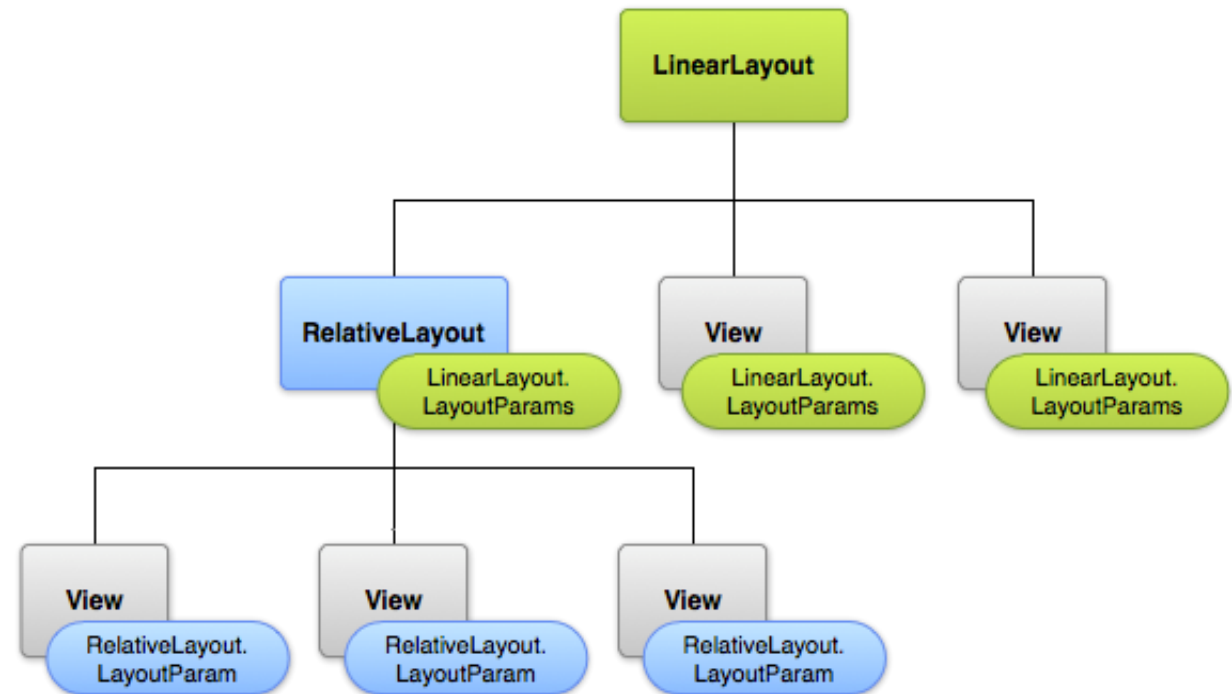
- The user interface is implemented as collection of view objects (e.g., a date widget, or editable text) –
- User's can construct sophisticated UIs by bundling views together using **layouts** (or **ViewGroups**) which can be considered as invisible containers.
- These containers can hold child containers. Each container defines its views (or other ViewGroups) and their layout properties.

✓ ממשק המשתמש ((UI בנדרואיד בנוי מקבוצות של View-ים, כגון

כפתורים, שדות טקסט, תמונות, רשימות ועוד.

✓ ניתן לבנות פריסות (Layouts) באמצעות XML או קוד Java.

✓ השימוש ב-Material Design מבטיח חוויית משתמש מודרנית ועקבית.



Linear Layout

- [LinearLayout](#) is a view group that aligns all children in a single direction, vertically or horizontally.
- You can specify the layout direction with the [android:orientation](#) attribute.
- All children of a [LinearLayout](#) are stacked one after the other,
- so a vertical list will only have one child per row, no matter how wide they are, and a horizontal list will only be one row high (the height of the tallest child, plus padding).
- A [LinearLayout](#) respects *margin*s between children and the *gravity*(right, center, or left alignment) of each child.

<https://developer.android.com/reference/android/widget/LinearLayout>

```
<LinearLayout ...  
    android:orientation="horizontal"  
    tools:context=".MainActivity">  
    <Button ... android:text="Button 1" />  
    <Button ... android:text="Button 2 Hooray" />  
    <Button ... android:text="Button 3" />  
    <Button ... android:text="Button 4  
        Very Long Text" />  
</LinearLayout>
```



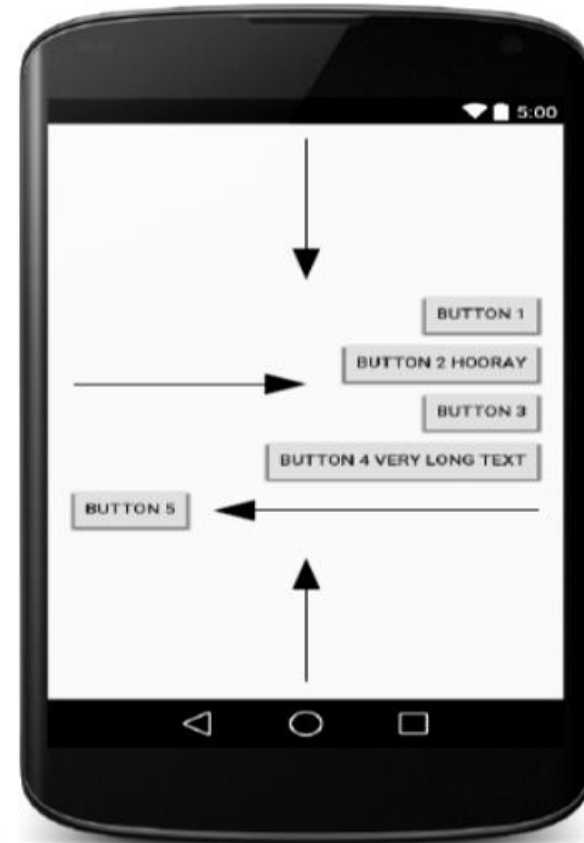
```
<LinearLayout ...  
    android:orientation="vertical"  
    tools:context=".MainActivity">  
    <Button ... android:text="Button 1" />  
    <Button ... android:text="Button 2  
        Hooray" />  
    <Button ... android:text="Button 3" />  
    <Button ... android:text="Button 4  
        Very Long Text" />  
</LinearLayout>
```



Gravity

- **gravity**: alignment direction that widgets are pulled
 - top, bottom, left, right, center
 - combine multiple with |
 - set **gravity** on the layout to adjust all widgets; set **layout_gravity** on an individual widget

```
<LinearLayout ...  
    android:orientation="vertical"  
    android:gravity="center|right">  
    <Button ... android:text="Button 1" />  
    <Button ... android:text="Button 2 Hooray" />  
    <Button ... android:text="Button 3" />  
    <Button ... android:text="Button 4 Very Long Text" />  
    <Button ... android:text="Button 5"  
                    android:layout_gravity="left" />  
</LinearLayout>
```

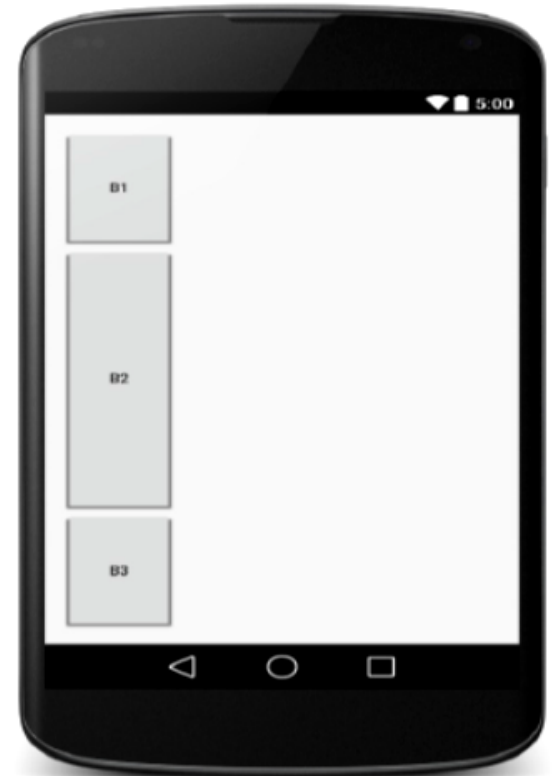


</LinearLayout> <https://developer.android.com/reference/android/widget/LinearLayout.LayoutParams#gravity>

Weight

- **weight**: gives elements relative sizes by integers
 - widget with weight **K** gets **K**/total fraction of total size
 - cooking analogy: "2 parts flour, 1 part water, ..."

```
<LinearLayout ...  
    android:orientation="vertical">  
    <Button ... android:text="B1"  
        android:layout_weight="1" />  
    <Button ... android:text="B2"  
        android:layout_weight="3" />  
    <Button ... android:text="B3"  
        android:layout_weight="1" />  
</LinearLayout>
```



Sizing an individual widget

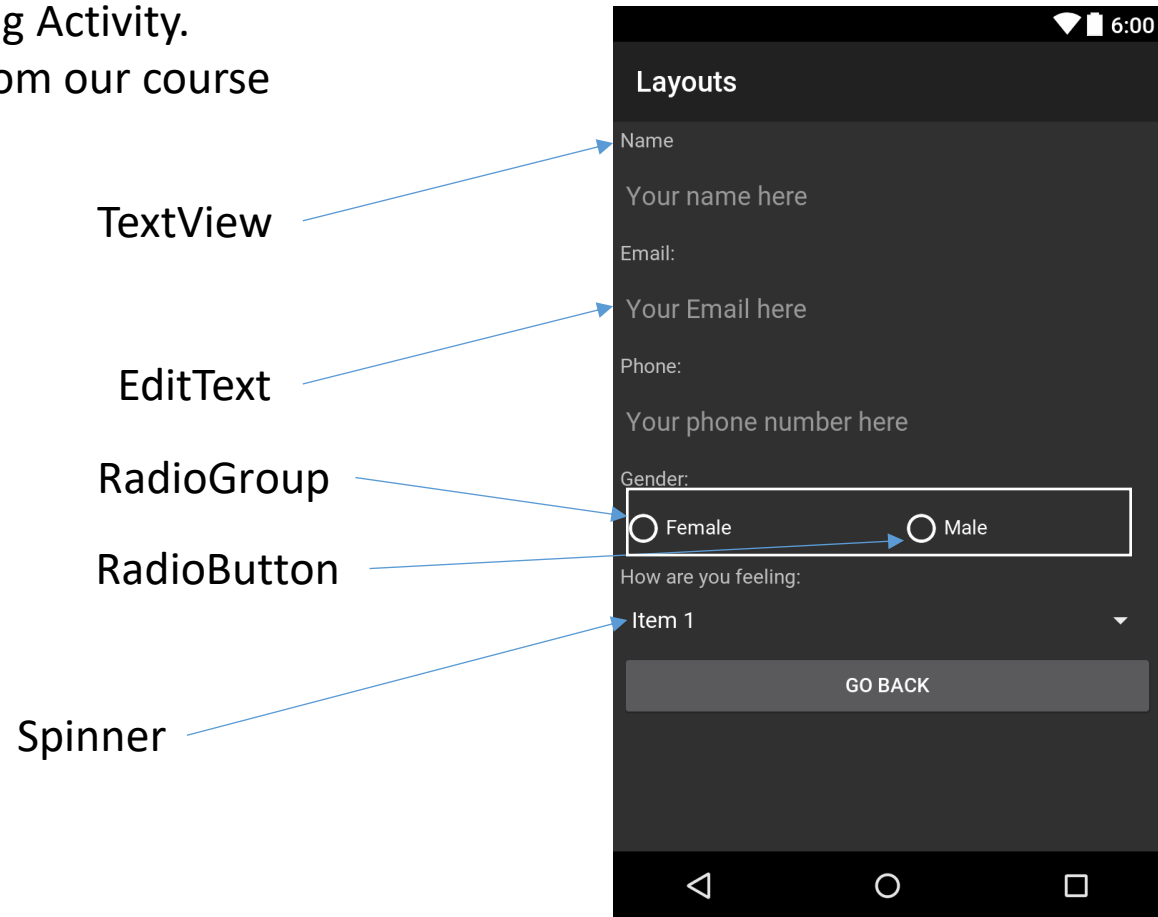
- **width** and **height** of a widget can be:
 - `wrap_content` : exactly large enough to fit the widget's content
 - `match_parent` : as wide or tall as 100% of the screen or layout

```
<Button ...  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content" />
```



LinearLayout example

Lets look at the code of the xml of the following Activity.
Import project called **layouts.zip** (download from our course website) into Android Studio



String resources

- Declare constant strings and arrays in `res/values/strings.xml`:

```
<resources>
    <string name="name">value</string>
    <string name="name">value</string>

    <string-array name="arrayname">
        <item>value</item>
        <item>value</item>
        <item>value</item>
        ...
        <item>value</item>
    </string-array>
</resources>
```

- Refer to them in Java code:

- as a resource: `R.string.name`, `R.array.name`
- as a string or array: `getResources().getString(R.string.name)`,
`getResources().getStringArray(R.array.name)`

- **Context:** Interface to global information about an application environment. This is an abstract class whose implementation is provided by the Android system. It allows access to application-specific resources and classes, as well as up-calls for application-level operations such as launching activities, broadcasting and receiving intents, etc.
- `getResources()`
 - is one of the methods of Context class:
 - A class whose implementation is provided by the Android system.
 - It allows access application-specific resources and classes call for application-level operations such as launching activities (e.g., `startActivity()`)

Dimensions of a phone used in laying out the UI

- When specifying the UI you need to be aware of a number of scaling units.
- **dpi == dots per inch.** The quantity of pixels within a physical area of the screen
- A low density screen has fewer pixels within a given physical area, compared to a medium or high density screens.
- **dp** == which stands for density independent pixel
- **1 dp == 1 pixel on a 160 dpi screen.**

מדוע זה חשוב?

✓ תמונות ואייקונים באפליקציה צריכים להתאים לצפיפות המסך כדי למנוע עיוותים וטשטוש.

✓ אנדרואיד מאפשרת שמירה של תמונות בגדלים שונים בתיקיות `res/drawable-mdpi`, `res/drawable-xhdpi` וכו'.

הבנת יחידות המידה באנדרואיד

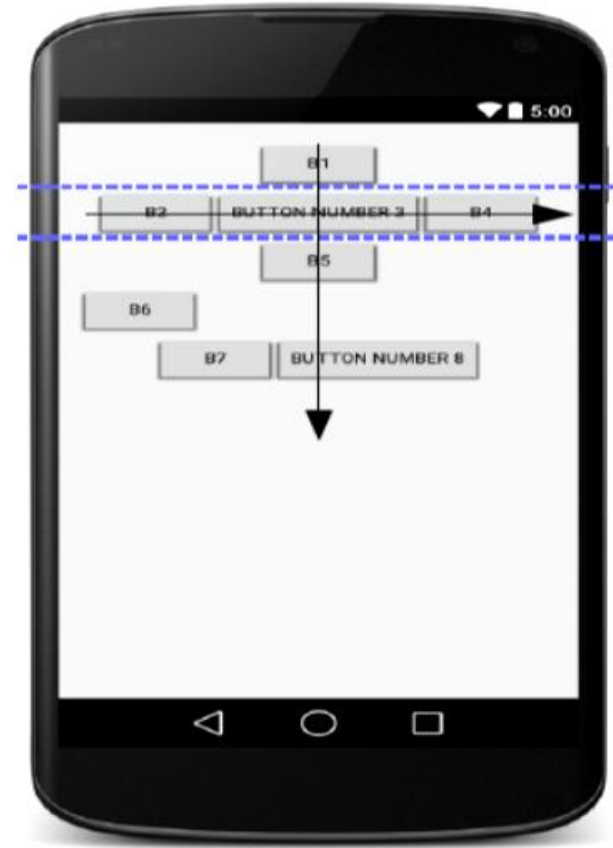
שימושים נפוצים	תיאור	יחידה
קביעת צפיפות המסך (low, medium, high, xhigh, xxhigh, xxxhigh)	מספר הפיקסלים באינץ' פיזי של המסך	dpi (dots per inch)
כל עיצוב ממשק (רוחב כפתורים, שוליים, תמונות)	פיקסלים שתואמים לכל המסכים לפי הצפיפות	dp (density-independent pixel)
עיצוב גופנים כדי לתמוך בגדלי טקסט משתנים	יחידה דומה ל-dp אך מותאמת לגודל טקסט דינמי	sp (scale-independent pixel)
עדיף להימנע מהם בעיצוב ממשק	פיקסלים רגילים, משתנים בין מסכים שונים	px (pixels)

טיפים לעיצוב נכון עם יחידות מידה

- השתמשו ב-dp לכלל המידות הוויזואליות (כפתורים, שוליים, תמונות).
- השתמשו ב-sp לכלל הטקסטים כדי לתמוך בשינוי גודל גופן.
- אל תשתמשו בפיקסלים (אפעיצוב ממשק, כדי למנוע בעיות התאמה.
- תמכו ברזולוציות שונות על ידי שמירת אייקונים ותמונות בכמה גדלים בתיקיות drawable-xxhdpi וכו'.

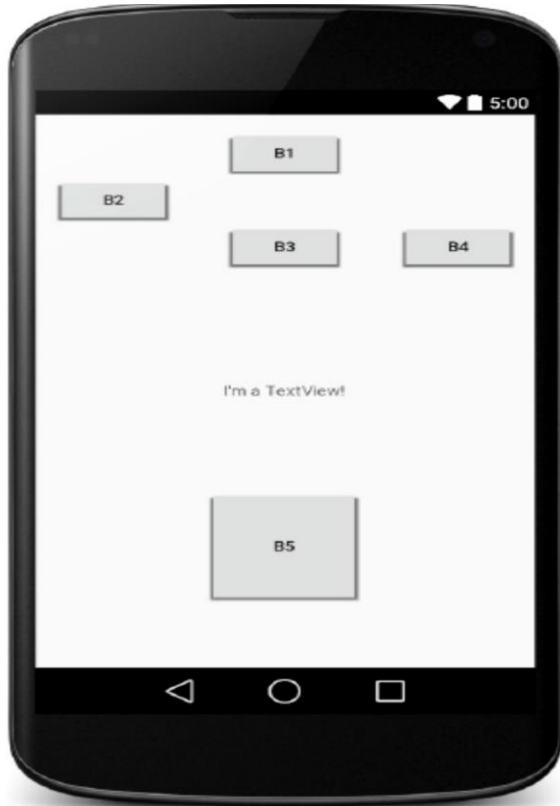
Nested layout solution

```
<LinearLayout ...  
    android:orientation="vertical" android:gravity="center|top">  
    <Button ... android:text="B1" />  
    <LinearLayout ...  
        android:layout_width="match_parent"  
        android:layout_height="wrap_content"  
        android:orientation="horizontal"  
        android:gravity="center|top">  
        <Button ... android:text="B2" />  
        <Button ... android:text="Button Number 3" />  
        <Button ... android:text="B4" />  
    </LinearLayout>  
    <Button ... android:text="B5" />  
    <Button ... android:text="B6" android:layout_gravity="left" />  
    <LinearLayout ...  
        android:layout_width="match_parent"  
        android:layout_height="wrap_content"  
        android:orientation="horizontal"  
        android:gravity="center|top">  
        <Button ... android:text="B7" />  
        <Button ... android:text="Button Number 8" />  
    </LinearLayout>  
</LinearLayout>
```



Relative Layout

- each widget's position and size are relative to other views
 - relative to "parent" (the activity itself)
 - relative to other widgets/views
 - x-positions of reference: left, right, center
 - y-positions of reference: top, bottom, center
- intended to reduce the need for nested layout



```
<RelativeLayout ... >
    <Button ... android:id="@+id/b1" android:text="B1"
        android:layout_alignParentTop="true"
        android:layout_centerHorizontal="true" />
    <Button ... android:id="@+id/b2" android:text="B2"
        android:layout_alignParentLeft="true"
        android:layout_below="@+id/b1" />
    <Button ... android:id="@+id/b3" android:text="B3"
        android:layout_centerHorizontal="true"
        android:layout_below="@+id/b2" />
    <Button ... android:id="@+id/b4" android:text="B4"
        android:layout_alignParentRight="true"
        android:layout_below="@+id/b2" />
    <TextView ... android:id="@+id/tv1"
        android:text="I'm a TextView!"
        android:layout_centerInParent="true" />
    <Button ... android:id="@+id/b5" android:text="B5"
        android:padding="50dp"
        android:layout_centerHorizontal="true"
        android:layout_alignParentBottom="true"
        android:layout_marginBottom="50dp" />
</RelativeLayout>
```

Relative anchor points

- properties for x/y relative to **another widget**:
 - **layout_below**, **above**, **toLeftOf**, **toRightOf**
 - set these to the ID of another widget in the format "**@id/*theID***"
(obviously, the given widget must have an ID for this to work)
- properties for x/y relative to layout **container** (the activity):
 - **layout_alignParentTop**, **Bottom**, **Left**, **Right**
 - set these flags to a boolean value of "true" to enable them
 - **layout_centerHorizontal**, **Vertical**, **InParent**
 - set these flags to "true" to center the control within its parent in a dimension

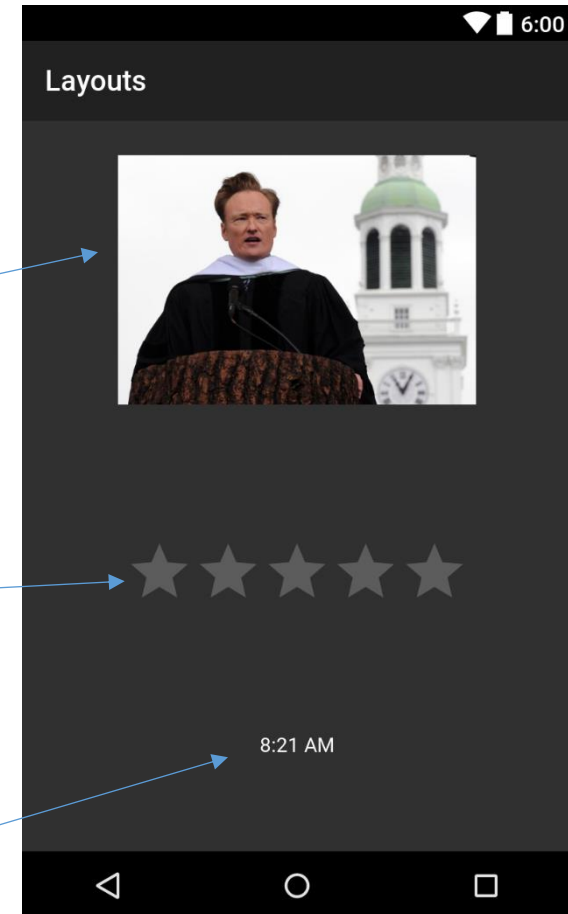
RelativeLayout example

Lets look at the code of the xml of the following Activity.
Import project called **layouts.zip** (download from our course website) into Android Studio

ImageView

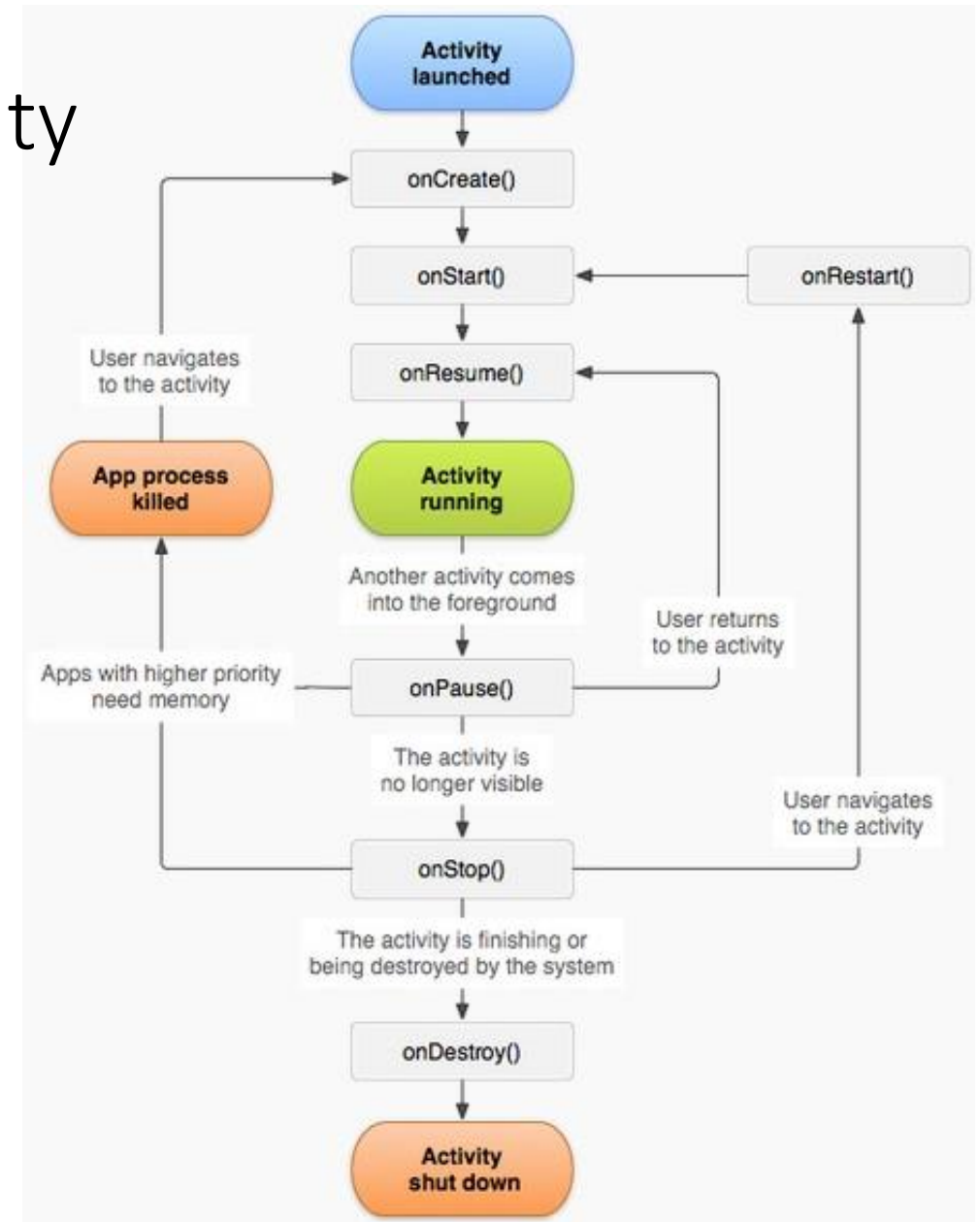
RatingBar

TextClock



Capturing the lifecycle of an activity

- An activity can be thought of as being in one of several states
- We want to use a necessary subset of these states so that our apps behave well when user and system driven events occur; The app **should not**:
 - **crash** when a user suddenly **receives a phone call**
 - **crash** when the user **switches app** (e.g., from Facebook to Google maps)
 - **lose the user's data or progress** if the user leaves and then later returns to the app
 - **crash or lose the user's progress** when the screen flips between **landscape** and **portrait orientation**.



Capturing the lifecycle of an activity

• מחזור החיים של Activity קובע כיצד האפליקציה מגיבה לאירועים מערכתיים ולפעולות המשתמש.

• שימוש נכון בשלבי המחזור מונע קריסות, אובדן מידע, וניהול לא יעיל של משאבי מערכת.

שלבי מחזור החיים של Activity:

- **onCreate** – יצירת ה-Activity והכנת ממשק המשתמש.
- **onStart** – ה-Activity הופכת לגלויה, אך עדיין לא פעילה.
- **onResume** – ה-Activity הופכת לאינטראקטיבית (מצב Foreground).
- **onPause** – המשתמש עובר למסך אחר, אך ה-Activity עדיין קיימת בזיכרון.
- **onStop** – ה-Activity כבר לא גלויה, המערכת עשויה לשחרר משאבים.
- **onDestroy** – ה-Activity נסגרת לצמיתות או כדי לפנות זיכרון.
- **onRestart** – מופעל כאשר ה-Activity חוזרת להיות פעילה אחרי **onStop**.

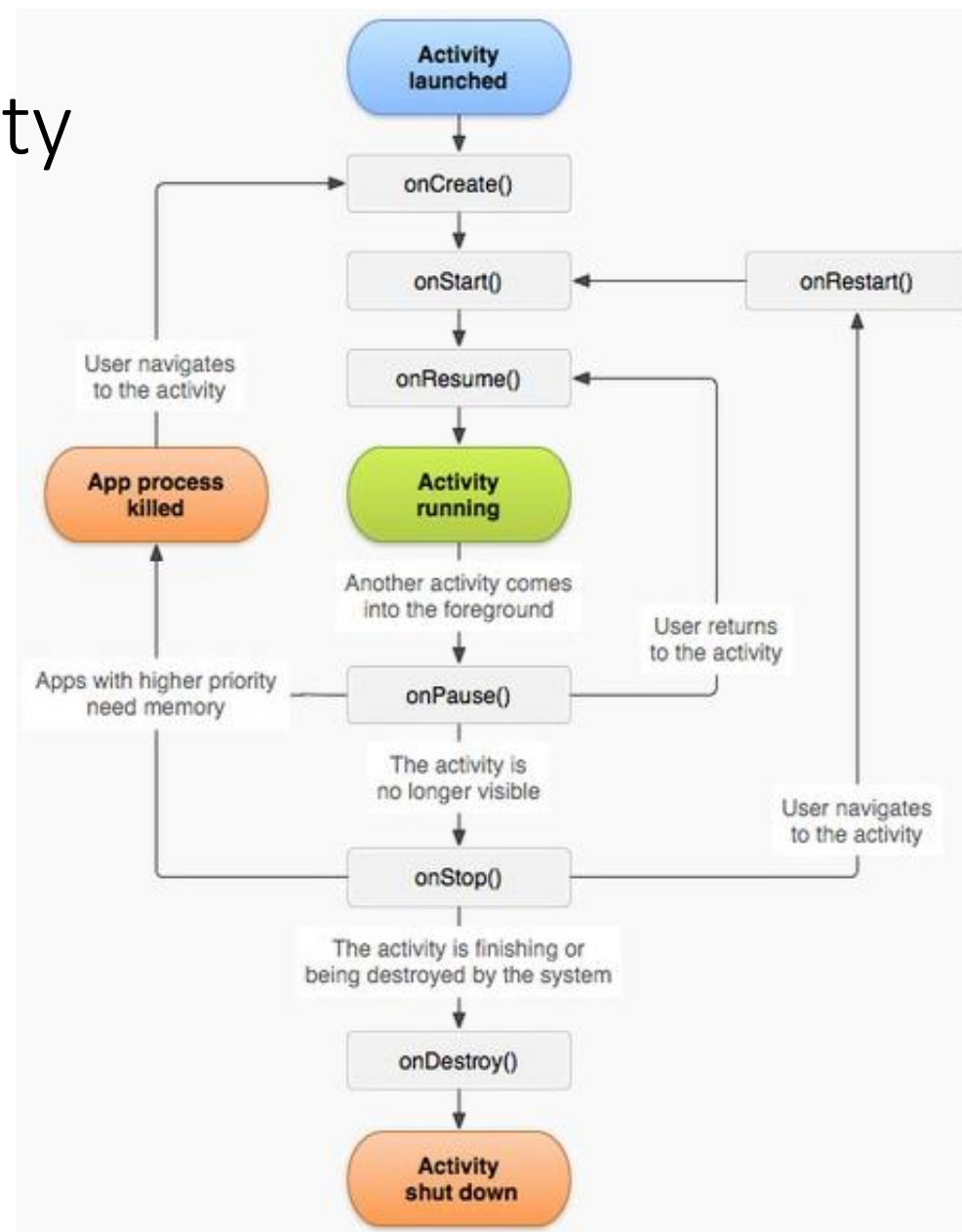
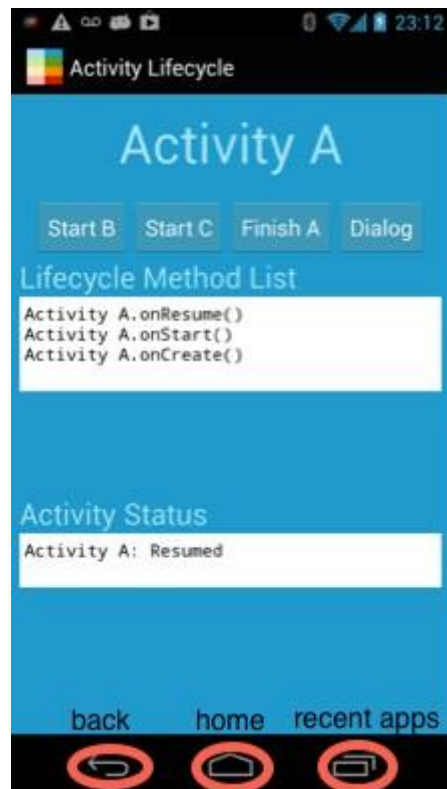


Illustration example for Activity lifecycle

- Download source code from course website (file name: activitya.zip)
- Import project into Android Studio

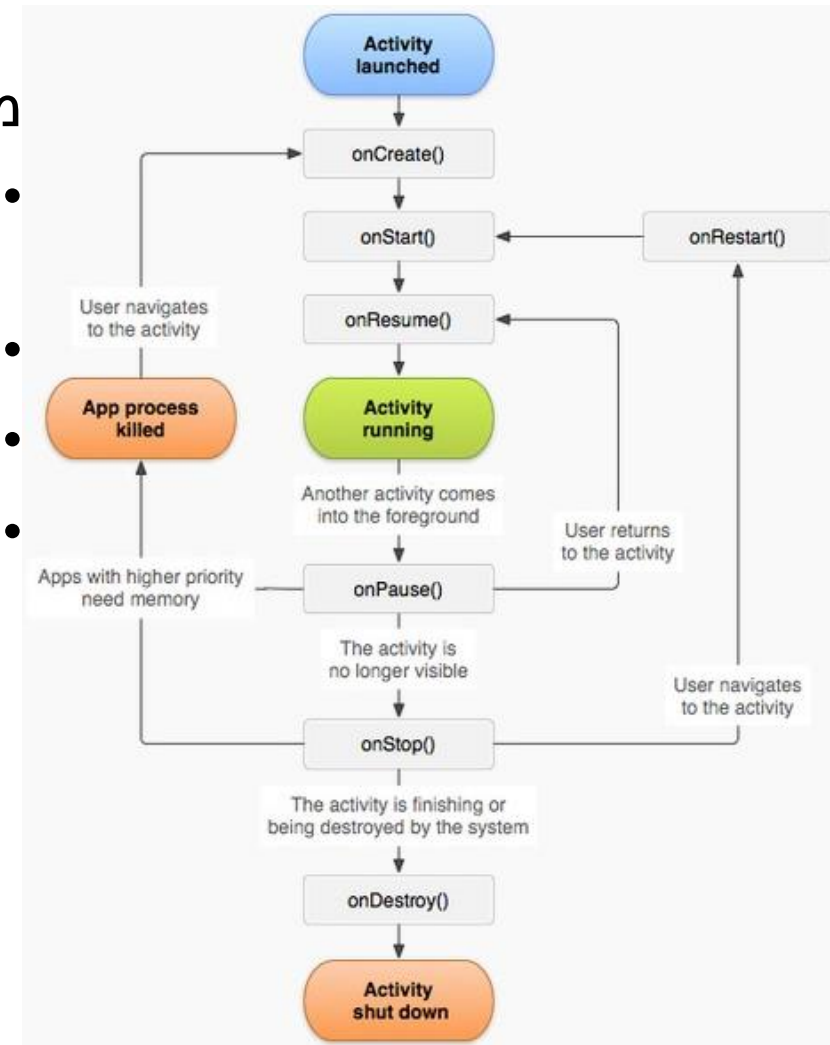


Create

מה קורה כאשר Activity נוצרת?

- כאשר Activity מופעלת, מערכת אנדרואיד קוראת – onCreate() השלב הראשון במחזור החיים של Activity.
- כאן נטענים משאבים סטטיים כמו תמונות, תפריטים ופריסות (Layouts).
- ה-UI מוגדר באמצעות setContentView().
- זהו המקום המתאים לאתחול משתנים ולבצע פעולות חיוניות לפני הצגת ה-Activity.

```
public class MyActivity extends Activity {  
    private Button okBtn;  
    public void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState); // always call super  
        setContentView(R.layout.my_activity); // set up layout  
        // Retain references to UI views as necessary  
        okBtn = (Button) findViewById(R.id.okbtn);  
    }  
}
```



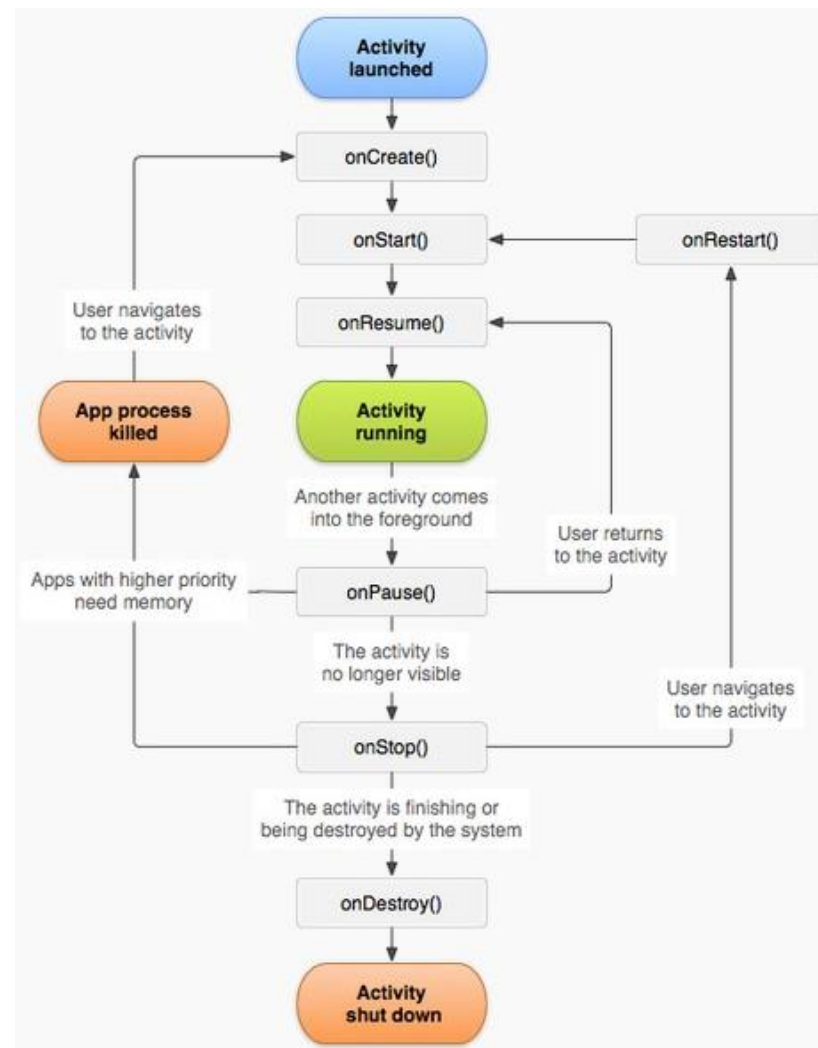
Destroy

- When **onDestroy** is called, your Activity is about to be destroyed
- **Clean up** any **resources** including ending threads, closing database connections etc...

מה קורה כאשר Activity מושמדת?

- **onDestroy()** מופעל כאשר ה Activity- מפסיקה לפעול לחלוטין ויוצאת ממחזור החיים.
- זה יכול לקרות משתי סיבות עיקריות:
- האפליקציה נסגרת על ידי המשתמש → לדוגמה, בלחיצה על כפתור "יציאה" שקורא ל- **finish()**.
- המערכת מחליטה לשחרר משאבים → כאשר יש עומס על הזיכרון וה- Activity אינה בשימוש.

```
public void onDestroy() {  
    super.onDestroy(); // always call super  
}
```



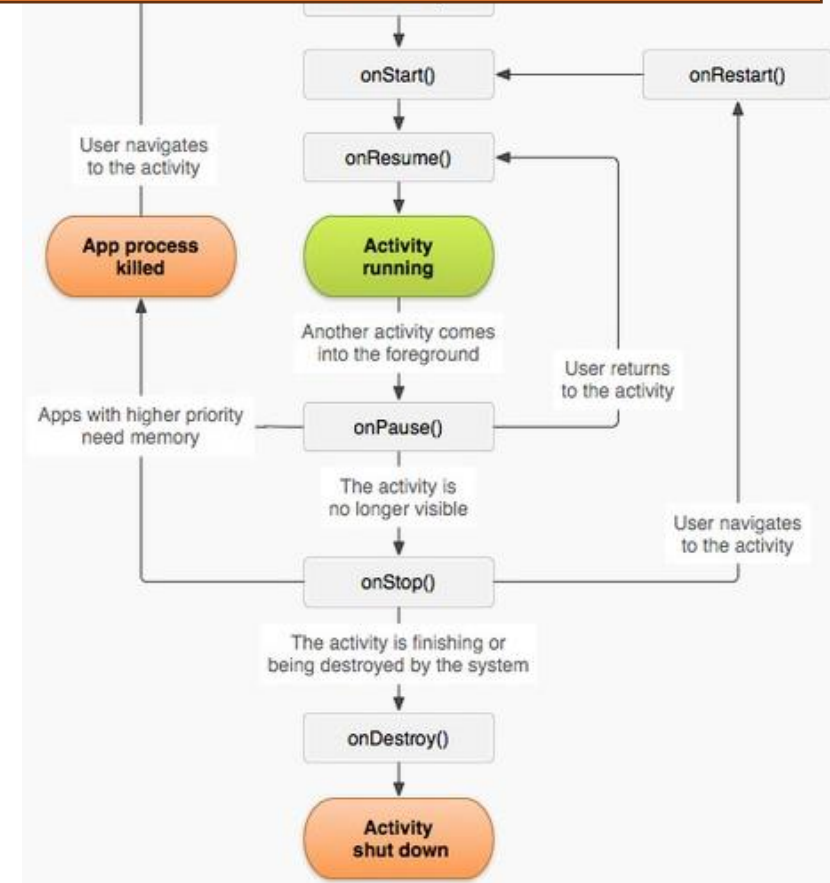
Resume

- This is a good place to
 - – **Initialize resources** that you will release in **onPause**.
 - – **open exclusive-access devices** (such as the camera), etc.
 - – **Start/resume animations** or other ongoing actions that should only run when activity is visible on screen.

```
public void onResume() {  
    super.onResume(); // always call super  
}
```

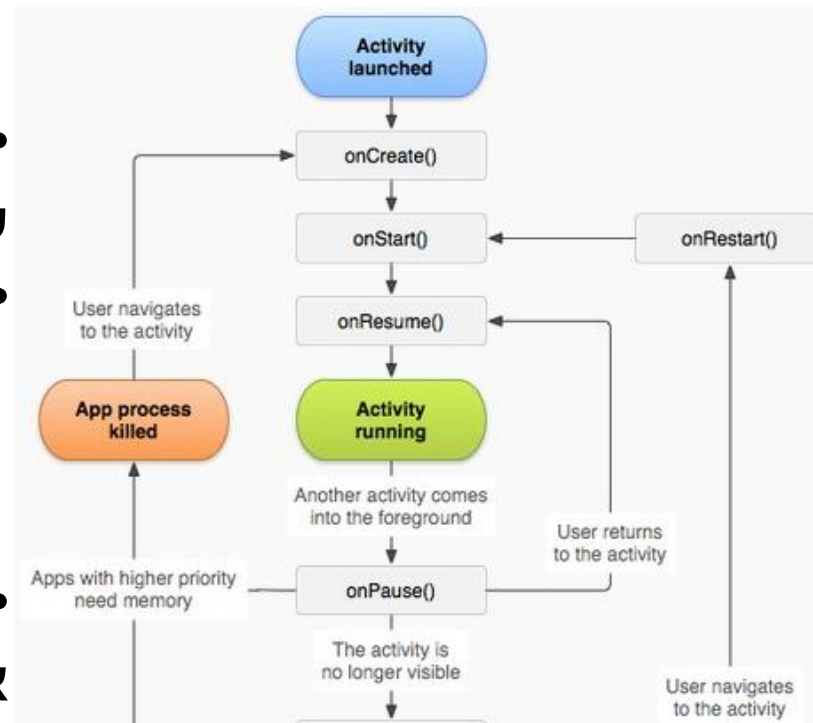
- **onResume()** נקראת כאשר ה **Activity**-חוזרת להיות פעילה לאחר שהמשתמש חזר אליה .
- זהו השלב שבו ה **Activity**-גלויה וניתן לבצע בה אינטראקציות .
- **ההבדל בין onStart() ל: onResume()**
- **onStart()** מתבצעת לפני שה **Activity**-הופכת לאינטראקטיבית .
- **onResume()** מתבצעת כשהאפליקציה מוכנה לשימוש מלא.

שימושים עיקריים ב**onResume()** –
אתחול משאבים שמשתמשים בהם רק כאשר המסך פעיל –
לדוגמה, הפעלת מצלמה או חיישנים.
חידוש אנימציות – אם הוקפאו ב**onPause()** –
רענון תוכן מהאינטרנט או מסד נתונים – עדכון מידע מתוך
השרת כאשר המשתמש חוזר ל-**Activity**.



Pause

- `onPause()` מופעלת כאשר ה-Activity **מאבדת פוקוס**, אך עדיין קיימת בזיכרון.
- זה קורה, למשל, כאשר:
 - המשתמש מקבל שיחה נכנסת.
 - נפתחת חלונית צפה (כמו דיאלוג).
 - המשתמש עובר לאפליקציה אחרת.
- **שלב ביניים לפני – `onStop()`** ה-Activity עדיין בזיכרון אך אינה פעילה.



שימושים עיקריים ב-`onPause()`

✓ שחרור משאבים שדורשים גישה בלעדית – למשל, השהיית המצלמה או הקלטת שמע.

✓ הפסקת תהליכים אינטנסיביים – כמו חישובים כבדים שמבזבזים סוללה.

✓ שמירת נתוני משתמש – כתיבה למסד נתונים או `SharedPreferences` כדי למנוע אובדן מידע.

@Override

```
protected void onPause() {
```

```
    super.onPause();
```

```
    mediaPlayer.pause(); // הפסקת השמעת מוזיקה
```

```
}
```

onStop

- When **onStop** is called, your activity is no longer visible on the screen:
 - – User chose another app from **Recent Apps** window.
 - – User starts a **different activity** in your app.
 - – User receives a **phone call** while in your app.

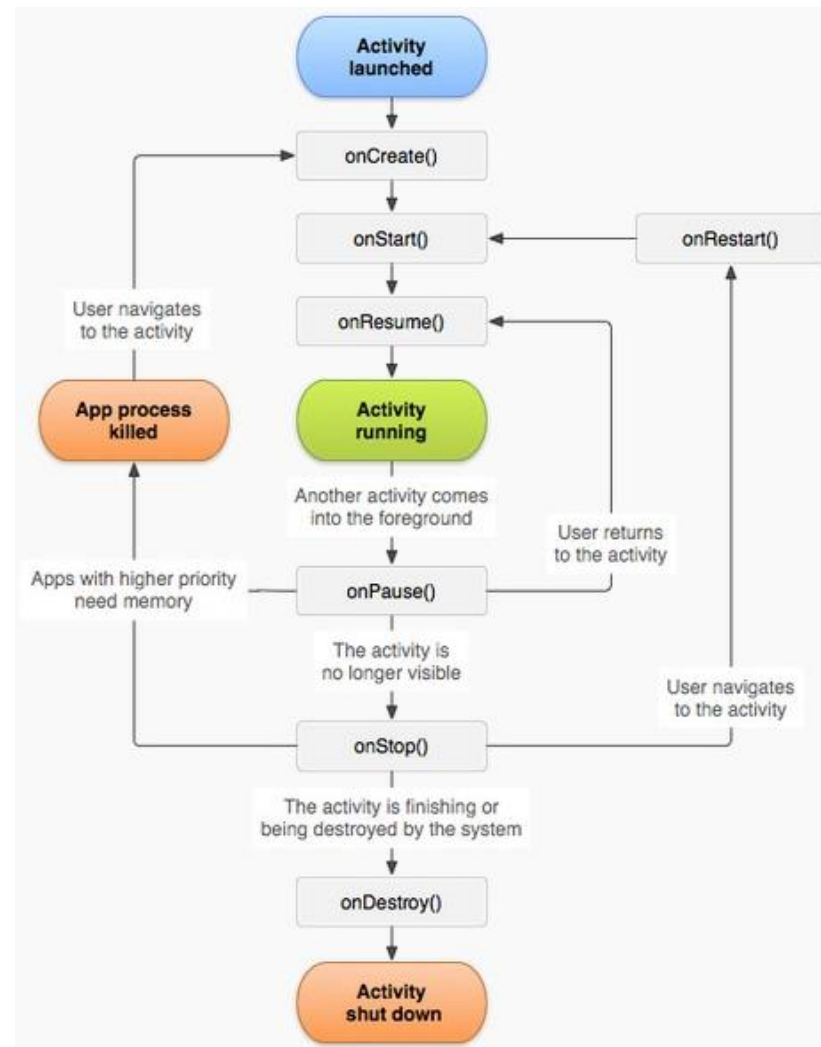
- `OnStop()` נקראת כאשר ה-`Activity` כבר אינה גלויה על המסך.
- קורה כאשר:

•המשתמש עובר לאפליקציה אחרת.

- ה-Activity מפנה מקום ל-Activity חדשה (לדוגמה, מעבר למסך אחר).
המערכת מחליטה להפסיק את ה-Activity כדי לפנות זיכרון.

@Override

```
public void onStop() {  
    // Always call the superclass method first  
    super.onStop();  
}
```



onRestart

- **onRestart** is called when the current activity is being re-displayed to the user (the user has navigated back to it)
 - – Reopen any resources that onStop closed.

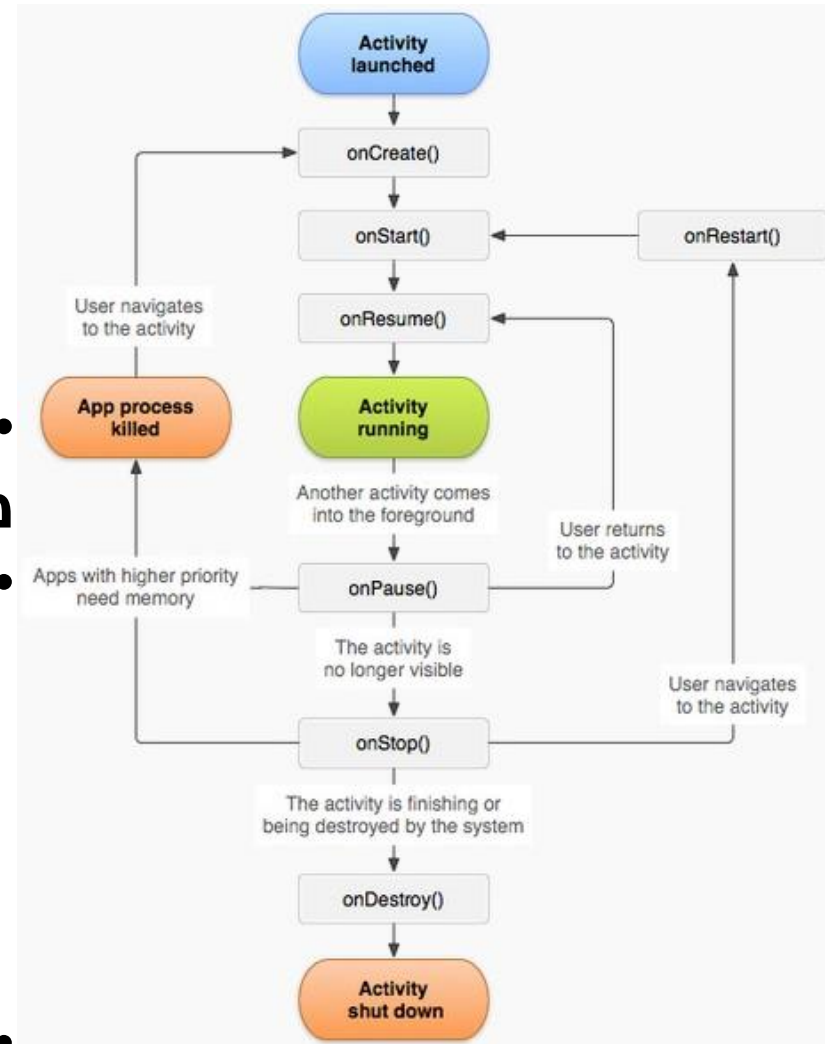
```
public void onRestart() {  
    super.onRestart(); // always call super  
}
```

- **onRestart()** נקראת כאשר Activity **שהייתה ב onStop()** מופעלת מחדש.

- זה קורה כאשר:

- המשתמש חוזר לאפליקציה דרך רשימת היישומים האחרונים. (Recent Apps)
- ה Activity-הופכת לגלויה מחדש לאחר שהוסתרה.
- המערכת נדרשה להשבית את ה Activity-כדי לפנות משאבים זמנית.

- **onRestart()** מתמיד מופעל לפני **onStart()** כאשר ה-Activity נטענה מחדש מהמצב הקודם.



Activity States that may killed by System

Likelihood of being killed	Process state	Activity state
Least	Foreground (having or about to get focus)	Created Started Resumed
More	Background (lost focus)	Paused
Most	Background (not visible)	Stopped
	Empty	Destroyed

מערכת אנדרואיד **מנהלת זיכרון באופן אוטומטי**, וכאשר יש מחסור בזיכרון, היא עשויה לסגור Activity כדי לפנות מקום לאפליקציות אחרות.

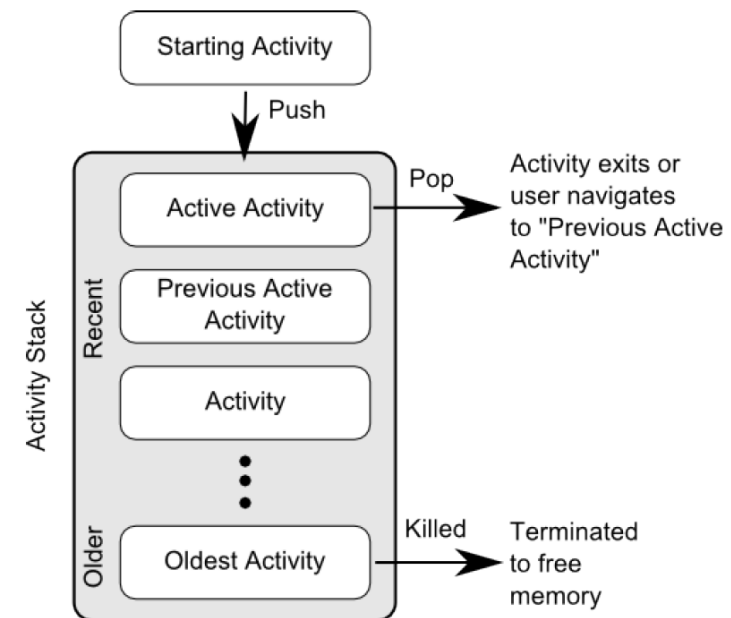
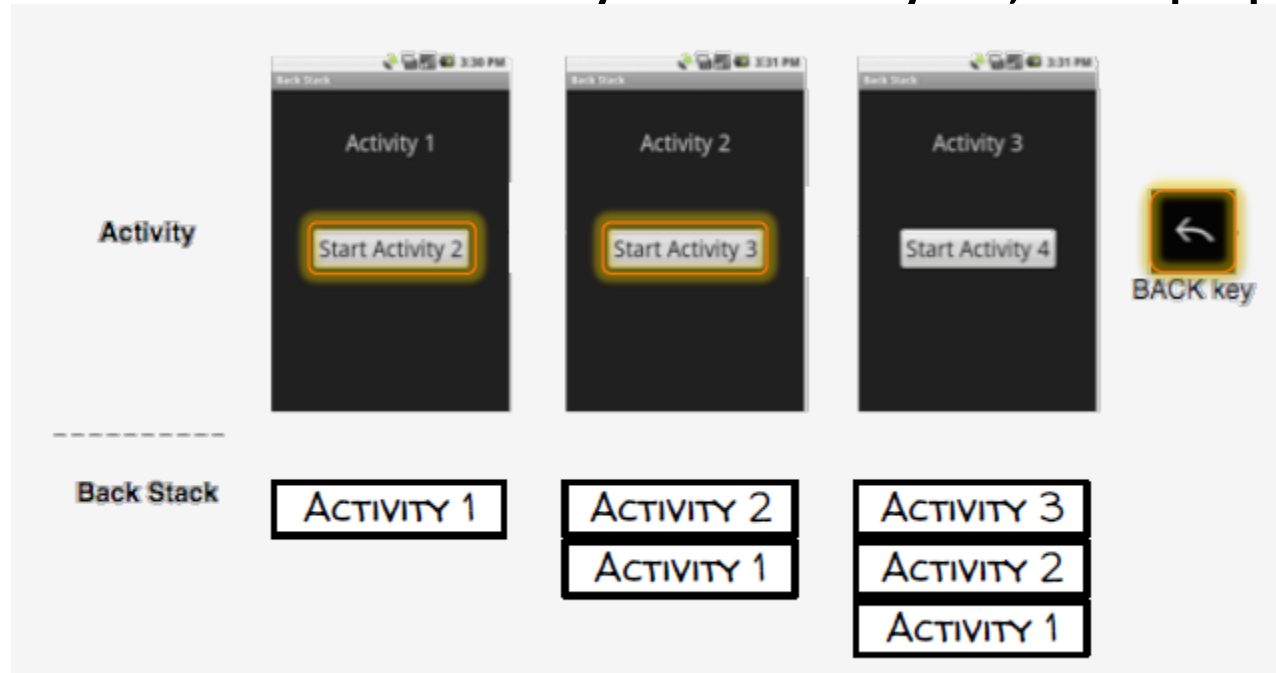
- Activity עלולה להיסגר בכל שלב בו היא אינה פעילה בקדמה.
- ככל שה Activity פחות פעילה, כך גדל הסיכוי שהמערכת תסגור אותה.

דוגמאות למצבים בהם Activity עלולה להיסגר

- ✓ **משתמש עבר לאפליקציה אחרת והשאיר את שלך ברקע** → onStop()
- מופעל והמערכת עשויה לסגור את התהליך אם יש עומס זיכרון.
- ✓ **מספר אפליקציות פתוחות בו זמנית** → אנדרואיד תעדיף לסגור Activity שלא נמצאת בקדמה כדי לפנות מקום.
- ✓ **שינוי כיוון המסך** → (Portrait ↔ Landscape) לעיתים המערכת הורסת ויוצרת מחדש את Activity כדי לטעון מחדש את ה-UI.

Birth, operation and death of an activity

- A Task is a set of related Activities
- These related activities don't have to be part of the same application
- When an Activity is launched, it goes on top of the **backstack**
- When the Activity is destroyed, it is popped off the **backstack**



Intent

- Apps have **multiple activities**.
 - An activity A can launch another activity B in response to an event.
 - The second activity B can send data back to A when it is done.
 - **Intent** is a way for one activity to invoke another
 - The activity can be in the same app or in a different app.
 - Can store extra data to pass as parameters to that activity.
 - Second activity can “return” information back to the caller if needed.

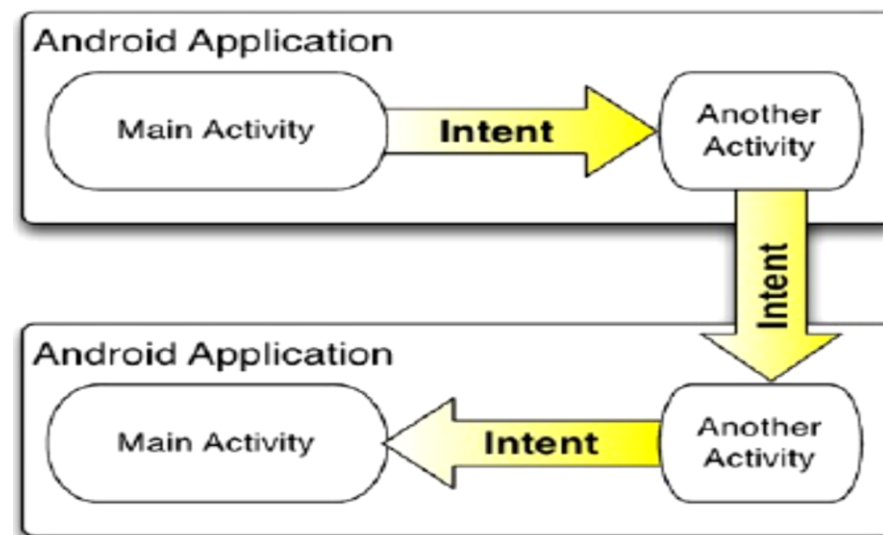
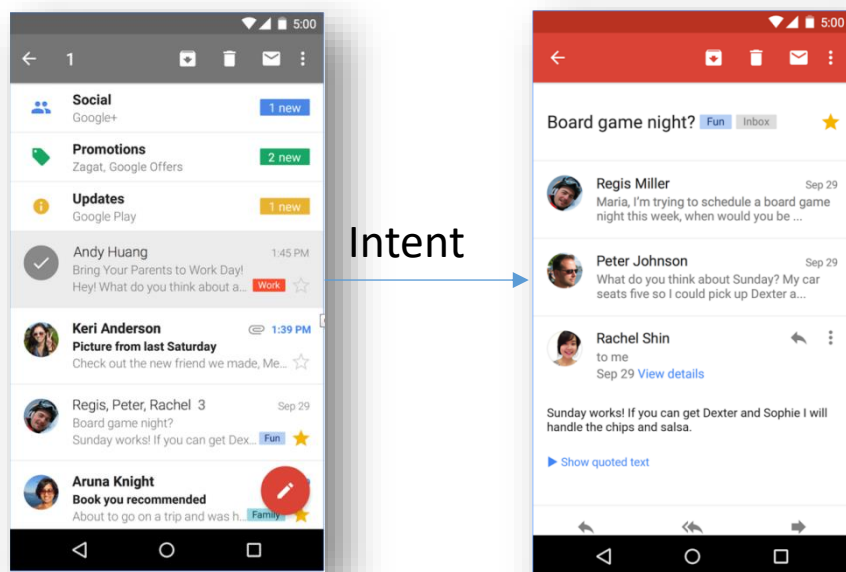
Intent הוא מנגנון להעברת מידע והפעלת רכיבים באפליקציה.

באמצעותו ניתן:

✓ להפעיל Activity חדשה מתוך Activity קיימת.

✓ לשלוח מידע בין רכיבים (Activities, Services, Broadcasts).

✓ להפעיל תהליכים במערכת (למשל, פתיחת מצלמה או דפדפן).



Explicit intents

Constructor: `Intent(Context ctx, Class<?> cls)`

Explicit Intent (אינטנט מפורש) משמש להפעלת רכיב ספציפי באפליקציה, כגון Activity, Service או BroadcastReceiver.

✓ מציין במפורש את שם הרכיב שיש להפעיל.

✓ משמש כאשר רוצים לעבור בין מסכים שונים באותה אפליקציה.

✓ מאפשר העברת נתונים בין Activities.

```
Intent intent = new Intent(this , SecondActivity.class);
startActivity(intent);
```

```
Intent intent = new Intent(this , SecondActivity.class);
Intent.putExtra("param1","abc");
Intent.putExtra("param2", 100);
startActivity(intent);
```

- In the second activity that was invoked, we can grab any extra data that was passed to it by the calling act.
 - Use getIntent() method to access the intent that spawned the activity
 - Use intent methods getExtra(), getIntExtra(), getStringExtra()... to extract the parameters values.

```
Public class SecondActivity extends Activity{
    ...
    public void onCreate(bundle savedInstanceState){
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_second);
        Intent intent = getIntent();
        String param1 = intent.getStringExtra("param1");
        int param2 = intent.getIntExtra("param2", -1);

    }
}
```

Get a result from an activity

כאשר מפעילים Activity נוספת עם Intent, לפעמים נרצה שהיא תחזיר נתונים ל Activity- שהפעילה אותה.

✓ משמש במקרים בהם המשתמש ממלא טופס, בוחר קובץ, מצלם תמונה, וכו'.
✓ בעבר השתמשו ב `startActivityForResult()` ו-`onActivityResult()` אך היום יש דרך מודרנית ומומלצת יותר.

- Activity results are used to return data from one activity to another.
- Common scenarios include selecting a file, taking a photo, or requesting permissions.
- Previously, developers used `startActivityForResult()` and `onActivityResult()` methods.
- These methods made the code complex due to scattered request codes, mixing result handling logic, and potential memory leaks because they weren't lifecycle-aware.

What is ActivityResultLauncher?

ActivityResultLauncher הוא הדרך המומלצת כיום להפעיל Activity אחרת ולקבל ממנה תוצאה, במקום `startActivityResult()`.
✓ יותר פשוט לניהול – אין צורך להשתמש ב-`requestCode`.
✓ מותאם למחזור החיים של ה-**Activity** – פחות בעיות זיכרון.
✓ קוד קריא וברור יותר.

- ActivityResultLauncher is a modern approach introduced as part of Android Jetpack to handle the result returned from another activity.
- It simplifies handling activity results through a more intuitive and declarative API, tightly integrated with the activity or fragment lifecycle.
- Eliminates ambiguity with request codes and makes the code less error-prone.
- Automatically tied to the component's lifecycle, preventing memory leaks by properly managing callbacks.

Example code snippet

```
public class MainActivity extends AppCompatActivity {
    private TextView resultTextView;
    private ActivityResultLauncher<Intent> secondActivityLauncher;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        resultTextView = findViewById(R.id.result_text_view);

        secondActivityLauncher = registerForActivityResult(
            new ActivityResultContracts.StartActivityForResult(),
            new ActivityResultCallback<ActivityResult>() {
                @Override
                public void onActivityResult(ActivityResult result) {
                    if (result.getResultCode() == RESULT_OK && result.getData() != null) {
                        String returnedString = result.getData().getStringExtra("result_string");
                        int returnedNumber = result.getData().getIntExtra("result_number", 0);
                        resultTextView.setText("Received string: " + returnedString + "\nReceived number: " + returnedNumber);
                    }
                }
            });

        findViewById(R.id.launch_second_activity_button).setOnClickListener(view -> {
            Intent intent = new Intent(MainActivity.this, SecondActivity.class);
            secondActivityLauncher.launch(intent);
        });
    }
}
```

שלב 1: יש לרשום את ה-Launcher בתחילת מחזור החיים של ה-Activity, בתוך onCreate().

שלב 1: יש להגדיר Callback שיטפל בתוצאה המוחזרת מה-Activity.

שלב 2: הפעלת ה-Activity עם בקשה לתוצאה ✓
כאשר המשתמש מבצע פעולה מסוימת (למשל לחיצה על כפתור), נשלח Intent להפעלת ה-Activity השנייה.

Example code snippet

```
import androidx.appcompat.app.AppCompatActivity;

public class SecondActivity extends AppCompatActivity {
    private EditText inputStringEditText, inputNumberEditText;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_second);

        inputStringEditText = findViewById(R.id.input_string);
        inputNumberEditText = findViewById(R.id.input_number);
        Button returnButton = findViewById(R.id.return_button);

        returnButton.setOnClickListener(view -> {
            Intent returnIntent = new Intent();
            String inputString = inputStringEditText.getText().toString();
            int inputNumber = Integer.parseInt(inputNumberEditText.getText().toString());
            returnIntent.putExtra("result_string", inputString);
            returnIntent.putExtra("result_number", inputNumber);
            setResult(RESULT_OK, returnIntent);
            finish();
        });
    }
}
```

שלב 3: החזרת תוצאה מה Activity- השנייה

✓ב Activity-השנייה, נשתמש ב-
setResult() כדי להחזיר נתונים
setResult(Activity.RESULT_OK,
intent); שולח את התוצאה.
finish(); סוגר את ה Activity-ומחזיר שליטה
ל Activity-הראשונה .

Implementation Step-by-Step

שלב 1: רישום ה-ActivityResultLauncher

✓ ייש לרשום את ה Launcher-בתחילת מחזור החיים של ה Activity-בתוך onCreate().

✓ ייש להגדיר Callback שיטפל בתוצאה המוחזרת מה Activity-.

✓ הרשמה ל ActivityResultLauncher-עם registerForActivityResult().

✓ טיפול בתוצאה המוחזרת בתוך ה Callback-.

שלב 2: הפעלת ה Activity-עם בקשה לתוצאה

✓ כאשר המשתמש מבצע פעולה מסוימת (למשל לחיצה על כפתור), נשלח Intent להפעלת ה Activity-השנייה.

activityResultLauncher.launch(intent); מחליף את startActivityResult().

✓ אין צורך בטיפול במספרי requestCode.

שלב 3: החזרת תוצאה מה Activity-השנייה

✓ ב Activity-השנייה, נשתמש ב setResult() כדי להחזיר נתונים

setResult(Activity.RESULT_OK, intent); שולח את התוצאה.

finish(); סוגר את ה Activity-ומחזיר שליטה ל Activity-הראשונה.

Real-World Use Case

- Example of capturing a photo to use in a profile update feature.

```
// Initialize the ActivityResultLauncher
```

```
cameraResultLauncher = registerForActivityResult(  
    new ActivityResultContracts.StartActivityForResult(),  
    new ActivityResultCallback<androidx.activity.result.ActivityResult>() {  
        @Override  
        public void onActivityResult(androidx.activity.result.ActivityResult result) {  
            if (result.getResultCode() == ComponentActivity.RESULT_OK) {  
                Bundle extras = result.getData().getExtras();  
                Bitmap imageBitmap = (Bitmap) extras.get("data");  
                mImageView.setImageBitmap(imageBitmap);  
            }  
        }  
    })  
);
```

```
private void openCamera() {  
    Intent takePictureIntent = new Intent(MediaStore.ACTION_IMAGE_CAPTURE);  
    if (takePictureIntent.resolveActivity(getPackageManager()) != null) {  
        cameraResultLauncher.launch(takePictureIntent);  
    } else {  
        Toast.makeText(this, "No application can handle this request. Please install",  
            Toast.LENGTH_LONG).show();  
    }  
}
```

AndroidManifest.xml

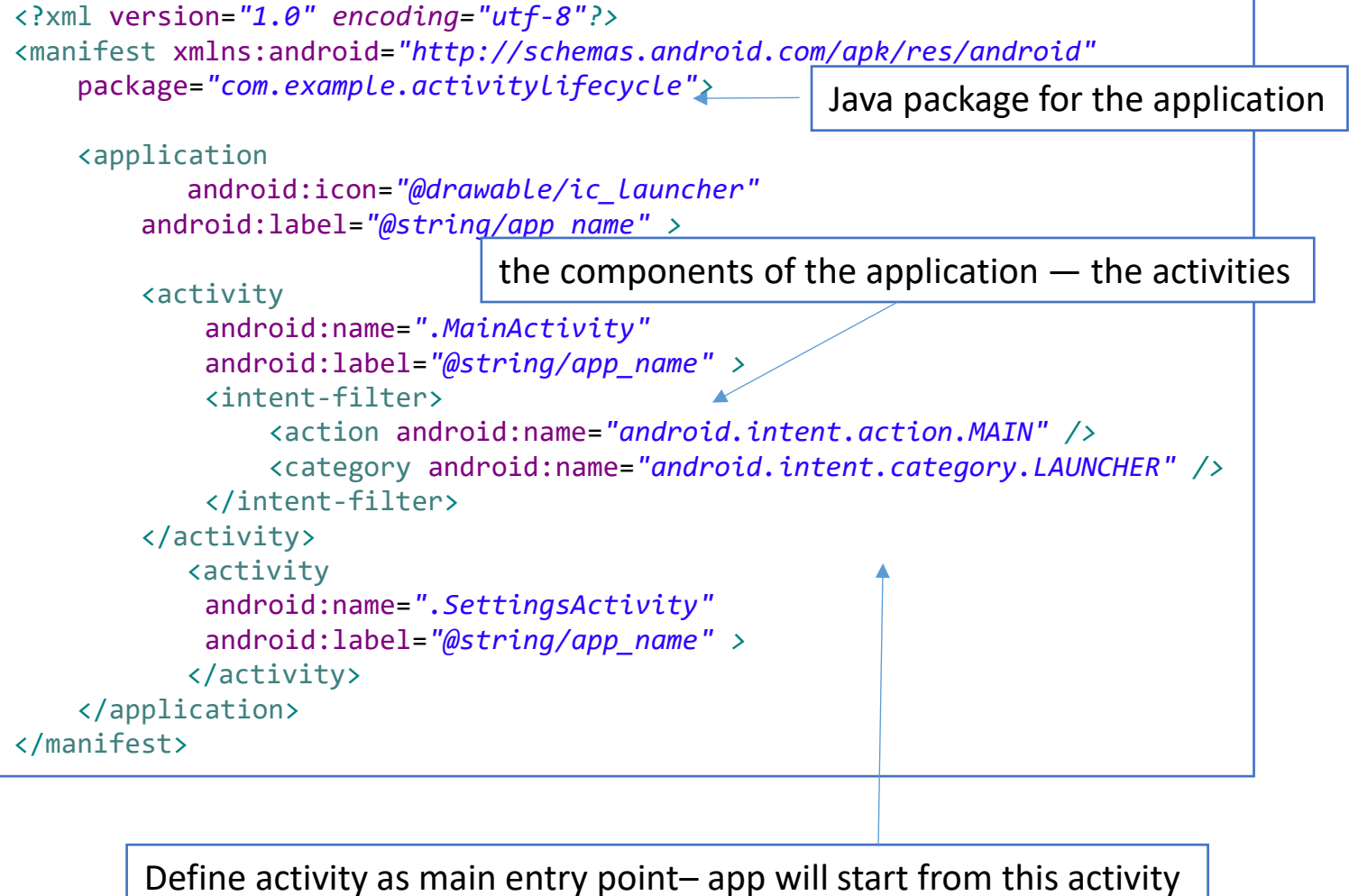
AndroidManifest.xml הוא קובץ הגדרות קריטי שנמצא בשורש של כל אפליקציית אנדרואיד.

✓ מספק למערכת מידע על הרכיבים של האפליקציה (Activities, Services, Broadcast Receivers).

✓ מגדיר הרשאות נדרשות (כגון גישה למצלמה, מיקום, קבצים).

✓ קובע נקודת כניסה ראשית לאפליקציה.

- Among other things, the manifest does the following:
 - names the Java package for the application. The package name serves as a unique identifier for the application.
 - describes the components of the application — the activities.....
 - declares which permissions the application must have in order to access protected parts.



<https://developer.android.com/guide/topics/manifest/manifest-intro>